In [5]:
```
import sys
sys.version
```

Out[5]:
```
'3.12.7 | packaged by Anaconda, Inc. | (main, Oct  4 2024, 13:17:27) [MSC v.192
9 64 bit (AMD64)]'
```

Variable

In [8]:
```
v = 5
v
```

Out[8]:
```
5
```

-Variable = IDENTIFIER = OBJECT variable name = value(value = datatype)- variable is case sensitive - var doesnot stsart with digit but end with digit - special charecter will not decalare as variable except _ - keyword reserved word - 35 reserve words | True, False, None - keyword always assigned with capital letter

In [12]:
```
nit = '7pm'
other
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[12], line 2
      1 nit = '7pm'
----> 2 other

NameError: name 'other' is not defined
```

In [14]:
```
nit = '7pm'
NIT
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[14], line 2
      1 nit = '7pm'
----> 2 NIT

NameError: name 'NIT' is not defined
```

In [16]:
```
nit = '7pm'
nit
```

Out[16]:
```
'7pm'
```

In [3]:
```
v5 =6
v5
```

Out[3]:
```
6
```

In [5]:
```
nit$ = 56
```

```
  Cell In[5], line 1
    nit$ = 56
       ^
SyntaxError: invalid syntax
```

In [7]:
```
x_train = 34
x_train
```

Out[7]:
```
34
```

In [21]:
```
import keyword
keyword.kwlist
```

Out[21]:
```
['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
 'class',
 'continue',
 'def',
 'del',
 'elif',
 'else',
 'except',
 'finally',
 'for',
 'from',
 'global',
 'if',
 'import',
 'in',
 'is',
 'lambda',
 'nonlocal',
 'not',
 'or',
 'pass',
 'raise',
 'return',
 'try',
 'while',
 'with',
 'yield']
```

In [23]:
```
len(keyword.kwlist)
```

Out[23]:
```
35
```

In [13]:
```
for = 67   # we dont use keyword as variable
for
```

```
  Cell In[13], line 1
    for = 67
        ^
SyntaxError: invalid syntax
```

In [15]:
```
For = 67
For
```

Out[15]: 67

In [17]:
```
false = 70
false
```

Out[17]: 70

python datatypes

1.Interger datatype

In [27]:
```
i = 56
i
```

Out[27]: 56

In [29]:
```
id(i)   #id= location
```

Out[29]: 140733004132504

In [31]:
```
True
```

Out[31]: True

In [33]:
```
False
```

Out[33]: False

In [37]:
```
false
```

Out[37]: 70

In [39]:
```
int(True)
```

Out[39]: 1

2. Float

In [48]:
```
a = 4.5
a
```

Out[48]: 4.5

In [52]:
```
4+10
```

Out[52]: 14

In [56]:
```
_
```

Out[56]: '{"dataframes": [], "user": "Lenovo"}'

In [58]:
```
_ + 5
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[58], line 1
----> 1 _ + 5

TypeError: can only concatenate str (not "int") to str
```

In [41]:
```
f3 = 2.5e2
f3
```

Out[41]: 250.0

In [45]:
```
f2 = 2e0
f2
```

Out[45]: 2.0

In [60]:
```
f4 = 2E0
f4
```

Out[60]: 2.0

3. boolean

In [62]:
```
True+True
```

Out[62]: 2

In [68]:
```
print(True+False)
print(True-False)
print(True-True)
print(False+False)
print(False-False)
```

```
1
1
0
0
0
```

In [70]:
```
10/2
```

Out[70]: 5.0

In [72]:
```
10//2
```

Out[72]: 5

In [74]:
```
False/True
```

Out[74]: 0.0

In [76]:
```
False//True
```

Out[76]: 0

In [78]:
```
True/False
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[78], line 1
----> 1 True/False

ZeroDivisionError: division by zero
```

4. Complex

In [86]:
```
c = 10 + 20j
c
d = 10 + 50j
d
```

Out[86]: (10+50j)

In [88]:
```
type(c)
```

Out[88]: complex

In [91]:
```
print(c)
print(d)
```

```
(10+20j)
(10+50j)
```

In [93]:
```
c + d
```

Out[93]: (20+70j)

In [95]:
```
print(type(i))
```

```
<class 'int'>
```

In [97]:
```
c.real
```

Out[97]: 10.0

In [99]:
```
c.imag
```

Out[99]: 20.0

5. String

In [102…
```
welcome = 7
welcome
```

Out[102… 7

In [108…
```
7 = welcome
7
```

```
  Cell In[108], line 1
    7 = welcome
    ^
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='?
```

In [110…
```
'welcome to college'
```

Out[110…
```
'welcome to college'
```

In [112…
```
"welcome to college"
```

Out[112…
```
'welcome to college'
```

In [114…
```
'''welcome to college'''
```

Out[114…
```
'welcome to college'
```

In [116…
```
'welcome
to
college'
```

```
  Cell In[116], line 1
    'welcome
    ^
SyntaxError: unterminated string literal (detected at line 1)
```

In [118…
```
'''welcome
to
college'''
```

Out[118…
```
'welcome \nto \ncollege'
```

In [120…
```
"welcome
to
college"
```

```
  Cell In[120], line 1
    "welcome
    ^
SyntaxError: unterminated string literal (detected at line 1)
```

Type Casting

In [4]:
```
int(230.34)    # float to interger
```

Out[4]:
```
230
```

In [6]:
```
int()
```

Out[6]:
```
0
```

In [10]:
```
int(230.34, 110.23)  # in float only one argument define  allow only one argumen
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[10], line 1
----> 1 int(230.34, 110.23)

TypeError: 'float' object cannot be interpreted as an integer
```

In [12]:
```
int(True)  # float to int
```

Out[12]: 1

In [14]:
```
int(False)
```

Out[14]: 0

In [16]:
```
int(1+2j) # comlex to int
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[16], line 1
----> 1 int(1+2j)

TypeError: int() argument must be a string, a bytes-like object or a real number,
not 'complex'
```

In [18]:
```
int('ten')   # string to int
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[18], line 1
----> 1 int('ten')

ValueError: invalid literal for int() with base 10: 'ten'
```

In [20]:
```
int('10')
```

Out[20]: 10

other datatypes to float type conversion

In [23]:
```
float(10)  # float to int
```

Out[23]: 10.0

In [25]:
```
float('welcome') # string to float
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[25], line 1
----> 1 float('welcome')

ValueError: could not convert string to float: 'welcome'
```

In [33]:
```
float('10')
```

Out[33]: 10.0

In [27]:
```
float(2+3j)    # complex to float
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[27], line 1
----> 1 float(2+3j)

TypeError: float() argument must be a string or a real number, not 'complex'
```

In [29]:
```
float(2.3)
```

Out[29]:
```
2.3
```

In [31]:
```
float(True)
```

Out[31]:
```
1.0
```

other data types to complex type conversion

In [36]:
```
complex(20) #  int to complex
```

Out[36]:
```
(20+0j)
```

In [38]:
```
complex(True)  # bool to complex
```

Out[38]:
```
(1+0j)
```

In [68]:
```
complex(False, False)
```

Out[68]:
```
0j
```

In [70]:
```
complex('10', '20')  # pass only one string argument
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[70], line 1
----> 1 complex('10', '20')

TypeError: complex() can't take second arg if first is a string
```

In [40]:
```
complex('hello')  # string to complex
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[40], line 1
----> 1 complex('hello')

ValueError: complex() arg is a malformed string
```

In [42]:
```
complex('10')
```

Out[42]:
```
(10+0j)
```

In [44]:
```
complex(2+3j)
```

Out[44]: (2+3j)

In [46]:
```
complex(10, 20)
```

Out[46]: (10+20j)

In [48]:
```
complex(10, 20, 30)  # coplex only take 2 arguments
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[48], line 1
----> 1 complex(10, 20, 30)

TypeError: complex() takes at most 2 arguments (3 given)
```

In [50]:
```
complex()
```

Out[50]: 0j

In [58]:
```
p, q = 20, 20
```

In [60]:
```
print(p)
print(q)
```

```
20
20
```

In [62]:
```
print(id(p))
print(id(q))
```

```
140733004131352
140733004131352
```

In [64]:
```
complex(10.3) # float to complex
```

Out[64]: (10.3+0j)

In [66]:
```
complex(10.3, 23)
```

Out[66]: (10.3+23j)

In [ ]:

In [ ]:

print()

In [4]:
```
num1 = 20
num2 = 30
```

In [12]:
```
num1, num2 = 20,30
num3 = num1+num2
print(num1)
print(num2)
print(num3)
print('The addition of' ,num1, 'and' ,num2,'is=',num3)
```

```
20
30
50
The addition of 20 and 30 is= 50
```

print with format method

In [26]:
```
num1 = 20
num2 = 30
add = num1+num2
print('The addition of {} and {} is= {}'.format(num1,num2,add))
```

```
The addition of 20 and 30 is= 50
```

print with end statement

In [30]:
```
print('hello')
print('how are you')
```

```
hello
how are you
```

In [34]:
```
print('hello' ,end=" ")
print('how are you')
```

```
hello how are you
```

Type casting

In [37]:
```
bool(1)
```

Out[37]:
```
True
```

In [39]:
```
bool(0)
```

Out[39]:
```
False
```

In [41]:
```
bool(0.1)
```

Out[41]:
```
True
```

In [43]:
```
bool( )
```

Out[43]:
```
False
```

In [45]:
```
bool(@)
```

```
  Cell In[45], line 1
    bool(@)
         ^
SyntaxError: invalid syntax
```

In [47]:
```
bool(1.2)
```

Out[47]:
```
True
```

In [49]:
```
bool(0.0)
```

Out[49]:
```
False
```

In [51]:
```
bool(1+2j)
```

Out[51]:
```
True
```

In [53]:
```
bool(2+)
```

```
  Cell In[53], line 1
    bool(2+)
           ^
SyntaxError: invalid syntax
```

In [55]:
```
bool('10')
```

Out[55]:
```
True
```

In [57]:
```
bool('ten')
```

Out[57]:
```
True
```

In [59]:
```
bool('zero')
```

Out[59]:
```
True
```

other datatype to string

In [62]:
```
str(1)
```

Out[62]:
```
'1'
```

In [66]:
```
str(10)
```

Out[66]:
```
'10'
```

In [68]:
```
str(2.0)
```

Out[68]:
```
'2.0'
```

In [70]:
```
int(True)
```

Out[70]:
```
1
```

In [72]:
```
str(False)
```

Out[72]:
```
'False'
```

In [74]:
```
str(1+2j)
```

Out[74]:
```
'(1+2j)'
```

In [ ]: